

§ 2.5 常规向量

2.5.1 直接引用元素

与数组直接通过下标访问元素的方式（形如“ $A[i]$ ”）相比，向量ADT所设置的`get()`和`put()`接口都显得不甚自然。毕竟，前一访问方式不仅更为我们所熟悉，同时也更加直观和便捷。那么，在经过封装之后，对向量元素的访问可否沿用数组的方式呢？答案是肯定的。

解决的方法之一就是重载操作符“`[]`”，具体实现如代码2.6所示。

```
1 template <typename T> T& Vector<T>::operator[](Rank r) const //重载下标操作符
2 { return _elem[r]; } // assert: 0 <= r < _size
```

代码2.6 重载向量操作符`[]`

2.5.2 置乱器

■ 置乱算法

可见，经重载后操作符“`[]`”返回的是对数组元素的引用，这就意味着它既可以取代`get()`操作（通常作为赋值表达式的右值），也可以取代`set()`操作（通常作为左值）。例如，采用这种形式，可以简明清晰地描述和实现如代码2.7所示的向量置乱算法。

```
1 template <typename T> void permute(Vector<T>& V) { //随机置乱向量，使各元素等概率出现于每一位置
2     for (int i = V.size(); i > 0; i--) //自后向前
3         swap(V[i - 1], V[rand() % i]); //V[i - 1]与V[0, i)中某一随机元素交换
4 }
```

代码2.7 向量整体置乱算法`permute()`

该算法从待置乱区间的末元素开始，逆序地向前逐一处理各元素。如图2.2(a)所示，对每一个当前元素 $V[i - 1]$ ，先通过调用`rand()`函数在 $[0, i)$ 之间等概率地随机选取一个元素，再令二者互换位置。注意，这里的交换操作`swap()`，隐含了三次基于重载操作符“`[]`”的赋值。

于是如图(b)所示，每经过一步这样的迭代，置乱区间都会向前拓展一个单元。因此经过 $O(n)$ 步迭代之后，即实现了整个向量的置乱。

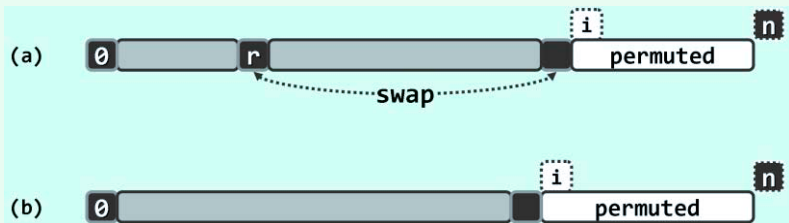


图2.2 向量整体置乱算法`permute()`的迭代过程

在软件测试、仿真模拟等应用中，随机向量的生成都是一项至关重要的基本操作，直接影响到测试的覆盖面或仿真的真实性。从理论上说，使用这里的算法`permute()`，不仅可以枚举出同一向量所有可能的排列，而且能够保证生成各种排列的概率均等（习题[2-6]）。

■ 区间置乱接口

为便于对各种向量算法的测试与比较，这里不妨将以上`permute()`算法封装至向量ADT中，并如代码2.8所示，对外提供向量的置乱操作接口`Vector::unsort()`。

```
1 template <typename T> void Vector<T>::unsort(Rank lo, Rank hi) { //等概率随机置乱向量区间[lo, hi)
2     T* V = _elem + lo; //将子向量_elem[lo, hi)视作另一向量V[0, hi - lo)
3     for (Rank i = hi - lo; i > 0; i--) //自后向前
4         swap(V[i - 1], V[rand() % i]); //将V[i - 1]与V[0, i)中某一元素随机交换
5 }
```

代码2.8 向量区间置乱接口`unsort()`

通过该接口，可以均匀地置乱任一向量区间`[lo, hi)`内的元素，故通用性有所提高。可见，只要将该区间等效地视作另一向量`V`，即可从形式上完整地套用以上`permute()`算法的流程。

尽管如此，还请特别留意代码2.7与代码2.8的细微差异：后者是通过下标，直接访问内部数组的元素；而前者则是借助重载的操作符“`[]`”，通过秩间接地访问向量的元素。

2.5.3 判等器与比较器

从算法的角度来看，“判断两个对象是否相等”与“判断两个对象的相对大小”都是至关重要的操作，它们直接控制着算法执行的分支方向，因此也是算法的“灵魂”所在。在本书中为了以示区别，前者多称作“比对”操作，后者多称作“比较”操作。当然，这两种操作之间既有联系也有区别，不能相互替代。比如，有些对象只能比对但不能比较；反之，支持比较的对象未必支持比对。不过，出于简化的考虑，在很多场合并不需要严格地将二者区分开来。

算法实现的简洁性与通用性，在很大程度上体现于：针对整数等特定数据类型的某种实现，可否推广至可比较或可比对的任何数据类型，而不必关心如何定义以及判定其大小或相等关系。若能如此，我们就可以将比对和比较操作的具体实现剥离出来，直接讨论算法流程本身。

为此，通常可以采用两种方法。其一，将比对操作和比较操作分别封装成通用的判等器和比较器。其二，在定义对应的数据类型时，通过重载“`<`”和“`==`”之类的操作符，给出大小和相等关系的具体定义及其判别方法。本书将主要采用后一方式。为节省篇幅，这里只给出涉及到的比较和判等操作符，读者可以根据实际需要，参照给出的代码加以扩充。

```
1 template <typename T> static bool lt(T* a, T* b) { return lt(*a, *b); } //less than
2 template <typename T> static bool lt(T& a, T& b) { return a < b; } //less than
3 template <typename T> static bool eq(T* a, T* b) { return eq(*a, *b); } //equal
4 template <typename T> static bool eq(T& a, T& b) { return a == b; } //equal
```

代码2.9 重载比较器以便比较对象指针

在一些复杂的数据结构中，内部元素本身的类型可能就是指向其它对象的指针；而从外部更多关注的，则往往是其所指对象的大小。若不加处理而直接根据指针的数值（即被指对象的物理地址）进行比较，则所得结果将毫无意义。

为此，这里不妨通过如代码2.9所示的机制，将这种情况与一般情况予以区分，并且约定在这种情况下，统一按照被指对象的大小做出判断。

2.5.4 无序查找

■ 判等器

代码2.1中`Vector::find(e)`接口，功能语义为“查找与数据对象`e`相等的元素”。这同时也暗示着，向量元素可通过相互比对判等——比如，元素类型`T`或为基本类型，或已重载操作符“`==`”或“`!=`”。这类仅支持比对，但未必支持比较的向量，称作无序向量（`unsorted vector`）。

■ 顺序查找

在无序向量中查找任意指定元素`e`时，因为没有更多的信息可以借助，故在最坏情况下——比如向量中并不包含`e`时——只有在访遍所有元素之后，才能得出查找结论。

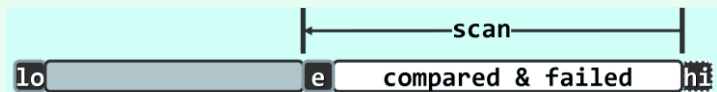


图2.3 无序向量的顺序查找

因此不妨如图2.3所示，从末元素起自后向前，逐一取出各个元素并与目标元素`e`进行比对，直至发现与之相等者（查找成功），或者直至检查过所有元素之后仍未找到相等者（查找失败）。这种依次逐个比对的查找方式，称作顺序查找（`sequential search`）。

■ 实现

针对向量的整体或区间，代码2.1分别定义了一个顺序查找操作的入口，其中前者作为特例，可直接通过调用后者而实现。因此，只需如代码2.10所示，实现针对向量区间的查找算法。

```
1 template <typename T> //无序向量的顺序查找：返回最后一个元素e的位置；失败时，返回lo - 1
2 Rank Vector<T>::find(T const& e, Rank lo, Rank hi) const { //assert: 0 <= lo < hi <= _size
3     while ((lo < hi--) && (e != _elem[hi])); //从后向前，顺序查找
4     return hi; //若hi < lo, 则意味着失败；否则hi即命中元素的秩
5 }
```

代码2.10 无序向量元素查找接口`find()`

其中若干细微之处，需要体会。比如，当同时有多个命中元素时，本书统一约定返回其中秩最大者——稍后介绍的查找接口`find()`亦是如此——故这里采用了自后向前的查找次序。如此，一旦命中即可立即返回，从而省略掉不必要的比对。另外，查找失败时约定统一返回`-1`。这不仅简化了对查找失败情况的判别，同时也使此时的返回结果更加易于理解——只要假想着在秩为`-1`处植入一个与任何对象都相等的哨兵元素，则返回该元素的秩当且仅当查找失败。

最后还有一处需要留意。`while`循环的控制逻辑由两部分组成，首先判断是否已抵达通配符，再判断当前元素与目标元素是否相等。得益于C/C++语言中逻辑表达式的短路求值特性，在前一判断非真后循环会立即终止，而不致因试图引用已越界的秩（`-1`）而出错。

■ 复杂度

最坏情况下，查找终止于首元素`_elem[lo]`，运行时间为 $O(hi - lo) = O(n)$ 。最好情况下，查找命中于末元素`_elem[hi - 1]`，仅需 $O(1)$ 时间。对于规模相同、内部组成不同的输入，渐进运行时间却有本质区别，故此类算法也称作输入敏感的（`input sensitive`）算法。

2.5.5 插入

■ 实现

按照代码2.1的ADT定义，插入操作`insert(r, e)`负责将任意给定的元素`e`插到任意指定的秩为`r`的单元。整个操作的过程，可具体实现如代码2.11所示。

```
1 template <typename T> //将e作为秩为r元素插入
2 Rank Vector<T>::insert(Rank r, T const& e) { //assert: 0 <= r <= size
3     expand(); //若有必要, 扩容
4     for (int i = _size; i > r; i--) _elem[i] = _elem[i-1]; //自后向前, 后继元素顺次后移一个单元
5     _elem[r] = e; _size++; //置入新元素并更新容量
6     return r; //返回秩
7 }
```

代码2.11 向量元素插入接口`insert()`

如前所述，插入之前必须首先调用`expand()`算法，核对是否即将溢出；若有必要（图2.4(a)），则加倍扩容（图2.4(b)）。

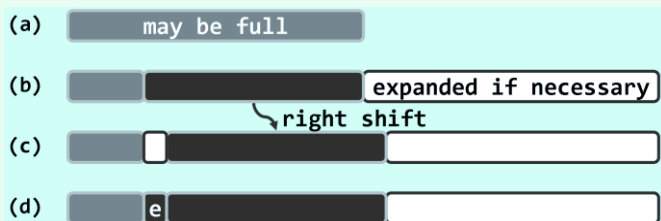


图2.4 向量元素插入操作`insert(r, e)`的过程

为保证数组元素物理地址连续的特性，随后需要将后缀`_elem[r, _size)`（如果非空）整体后移一个单元（图(c)）。这些后继元素自后向前的搬迁次序不能颠倒，否则会因元素被覆盖而造成数据丢失。在单元`_elem[r]`腾出之后，方可将待插入对象`e`置入其中（图(d)）。

■ 复杂度

时间主要消耗于后继元素的后移，线性正比于后缀的长度，故总体为 $O(\text{size} - r + 1)$ 。

可见，新插入元素越靠后（前）所需时间越短（长）。特别地，`r`取最大值`_size`时为最好情况，只需 $O(1)$ 时间；`r`取最小值`0`时为最坏情况，需要 $O(\text{size})$ 时间。一般地，若插入位置等概率分布，则平均运行时间为 $O(\text{size}) = O(n)$ （习题[2-9]），线性正比于向量的实际规模。

2.5.6 删除

删除操作重载有两个接口，`remove(lo, hi)`用以删除区间`[lo, hi)`内的元素，而`remove(r)`用以删除秩为`r`的单个元素。乍看起来，利用后者即可实现前者：令`r`从`hi - 1`到`lo`递减，反复调用`remove(r)`。不幸的是，这一思路似是而非。

因数组中元素的地址必须连续，故每删除一个元素，所有后继元素都需向前移动一个单元。若后继元素共有 $m = \text{size} - hi$ 个，则对`remove(r)`的每次调用都需移动 m 次；对于整个区间，元素移动的次数累计将达到 $m \cdot (hi - lo)$ ，为后缀长度和待删除区间宽度的乘积。

实际可行的思路恰好相反，应将单元素删除视作区间删除的特例，并基于后者来实现前者。稍后就会看到，如此可将移动操作的总次数控制在 $O(m)$ 以内，而等待删除区间的宽度无关。

■ 区间删除: `remove(lo, hi)`

向量区间删除接口`remove(lo, hi)`，可实现如代码2.12所示。

```
1 template <typename T> int Vector<T>::remove(Rank lo, Rank hi) { //删除区间[lo, hi)
2     if (lo == hi) return 0; //出于效率考虑，单独处理退化情况，比如remove(0, 0)
3     while (hi < _size) _elem[lo++] = _elem[hi++]; //[hi, _size)顺次前移hi - lo个单元
4     _size = lo; //更新规模，直接丢弃尾部[lo, _size = hi)区间
5     shrink(); //若有必要，则缩容
6     return hi - lo; //返回被删除元素的数目
7 }
```

代码2.12 向量区间删除接口`remove(lo, hi)`

设 $[lo, hi)$ 为向量(图2.5(a))的合法区间(图(b))，则其后缀 $[hi, n)$ 需整体前移 $hi - lo$ 个单元(图(c))。与插入算法同理，这里后继元素自前向后的移动次序也不能颠倒(习题[2-10])。

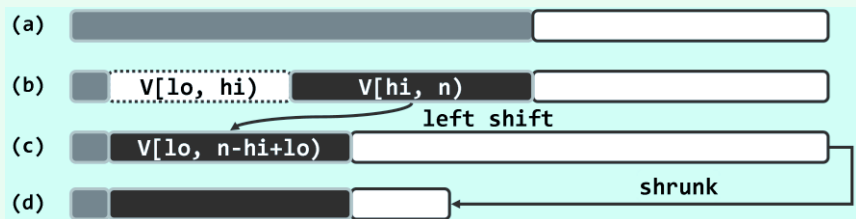


图2.5 向量区间删除操作`remove(lo, hi)`的过程

向量规模更新为`_size - hi + lo`后，还要调用`shrink()`，若有必要则做缩容处理(图(d))。

■ 单元素删除`remove(r)`

利用以上`remove(lo, hi)`通用接口，通过重载即可实现如下另一同名接口`remove(r)`。

```
1 template <typename T> T Vector<T>::remove(Rank r) { //删除向量中秩为r的元素, 0 <= r < size
2     T e = _elem[r]; //备份被删除元素
3     remove(r, r + 1); //调用区间删除算法，等效于对区间[r, r + 1)的删除
4     return e; //返回被删除元素
5 }
```

代码2.13 向量单元素删除接口`remove()`

■ 复杂度

`remove(lo, hi)`的计算成本，主要消耗于后续元素的前移，线性正比于后缀的长度，总体不过 $O(m + 1) = O(_size - hi + 1)$ 。

这与此前的预期完全吻合：区间删除操作所需的时间，应该仅取决于后继元素的数目，而与删除区间本身的宽度无关。特别地，基于该接口实现的单元素删除接口`remove(r)`需耗时 $O(_size - r)$ 。也就是说，被删除元素在向量中的位置越靠后（前）所需时间越短（长），最好为 $O(1)$ ，最坏为 $O(n) = O(_size)$ 。

■ 错误及意外处理

请注意，上述操作接口对输入都有一定的限制和约定。其中指定的待删除区间，必须落在合法范围 $[0, _size)$ 之内，为此输入参数必须满足 $0 \leq lo \leq hi \leq _size$ 。

一般地，输入参数超出接口所约定合法范围的此类问题，都属于典型的错误（**error**）或意外（**exception**）。除了以注释的形式加以说明，还应该尽可能对此类情况做更为周全的处理。

尽管如此，本书还是沿用了相对简化的处置方式，将入口参数合法性检查的责任统一交由上层调用例程，这也是出于对本书的重点——算法效率、讲解重点以及叙述简洁——的优先考虑。当然，在充分掌握了本书的内容之后，读者不妨再按照软件工程的规范，就此做进一步的完善。

2.5.7 唯一化

很多应用中，在进一步处理之前都要求数据元素互异。以网络搜索引擎为例，多个计算节点各自获得的局部搜索结果，需首先剔除其中重复的项目，方可合并为一份完整的报告。类似地，所谓向量的唯一化处理，就是剔除其中的重复元素，即表2.1所列deduplicate()接口的功能。

■ 实现

视向量是否有序，该功能有两种实现方式，以下首先介绍针对无序向量的唯一化算法。

```
1 template <typename T> int Vector<T>::deduplicate() { //删除无序向量中重复元素（高效版）
2     int oldSize = _size; //记录原规模
3     Rank i = 1; //从_elem[1]开始
4     while (i < _size) //自前向后逐一考查各元素_elem[i]
5         (find(_elem[i], 0, i) < 0) ? //在其前缀中寻找与之雷同者（至多一个）
6         i++ : remove(i); //若无雷同则继续考查其后继，否则删除雷同者
7     return oldSize - _size; //向量规模变化量，即被删除元素总数
8 }
```

代码2.14 无序向量清除重复元素接口deduplicate()

如代码2.14所示，该算法自前向后逐一考查各元素_elem[i]，并通过调用find()接口，在其前缀中寻找与之雷同者。若找到，则随即删除；否则，转而考查当前元素的后继。

■ 正确性

算法的正确性由以下不变性保证：

在while循环中，在当前元素的前缀_elem[0, i)内，所有元素彼此互异

初次进入循环时 $i = 1$ ，只有唯一的前驱_elem[0]，故不变性自然满足。



图2.6 无序向量deduplicate()算法原理

一般地如图2.6(a)所示，假设在转至元素 $e = _elem[i]$ 之前不变性一直成立。于是，经过

针对该元素的一步迭代之后，无非两种结果：

1) 若元素 e 的前缀`_elem[0, i)`中不含与之雷同的元素，则如图(b)，在做过`i++`之后，新的前缀`_elem[0, i)`将继续满足不变性，而且其规模增加一个单位。

2) 反之，若含存在与 e 雷同的元素，则由此前一直满足的不变性可知，这样的雷同元素不超过一个。因此如图(c)，在删除 e 之后，前缀`_elem[0, i)`依然保持不变性。

■ 复杂度

由图2.6(a)和(b)也可看出该算法过程所具有的单调性：

随着循环的不断进行，当前元素的后继持续地严格减少

因此，经过 $n - 2$ 步迭代之后该算法必然终止。

这里所需的时间，主要消耗于`find()`和`remove()`两个接口。根据2.5.4节的结论，前一部分时间应线性正比于查找区间的宽度，即前驱的总数；根据2.5.6节的结论，后一部分时间应线性正比于后继的总数。因此，每步迭代所需时间为 $O(n)$ ，总体复杂度应为 $O(n^2)$ 。

经预排序转换之后，借助2.6.3节将要介绍的相关算法，还可以进一步提高向量唯一化处理的效率（习题[2-12]）。

2.5.8 遍历

■ 功能

在很多算法中，往往需要将向量作为一个整体，对其中所有元素实施某种统一的操作，比如输出向量中的所有元素，或者按照某种运算流程统一修改所有元素的数值（习题[2-13]）。针对此类操作，可为向量专门设置一个遍历接口`traverse()`。

■ 实现

向量的遍历操作接口，可实现如代码2.15所示。

```
1 template <typename T> void Vector<T>::traverse(void (*visit)(T&)) //利用函数指针机制的遍历
2 { for (int i = 0; i < _size; i++) visit(_elem[i]); }
3
4 template <typename T> template <typename VST> //元素类型、操作器
5 void Vector<T>::traverse(VST& visit) //利用函数对象机制的遍历
6 { for (int i = 0; i < _size; i++) visit(_elem[i]); }
```

代码2.15 向量遍历接口`traverse()`

可见，`traverse()`遍历的过程，实质上就是自前向后地逐一对各元素实施同一基本操作。而具体采用何种操作，可通过两种方式指定。前一种方式借助函数指针`*visit()`指定某一函数，该函数只有一个参数，其类型为对向量元素的引用，故通过该函数即可直接访问或修改向量元素。另外，也可以函数对象的形式，指定具体的遍历操作。这类对象的操作符“`()`”经重载之后，在形式上等效于一个函数接口，故此得名。

相比较而言，后一形式的功能更强，适用范围更广。比如，函数对象的形式支持对向量元素的关联修改。也就是说，对各元素的修改不仅可以相互独立地进行，也可以根据某个（些）元素的数值相应地修改另一元素。前一形式虽也可实现这类功能，但要繁琐很多。

■ 实例

在代码2.16中，`Increase<T>()`即是按函数对象形式指定的基本操作，其功能是将作为参数的引用对象的数值加一（假定元素类型`T`可直接递增或已重载操作符“++”）。于是可如`increase()`函数那样，以此基本操作做遍历即可使向量内所有元素的数值同步加一。

```
1 template <typename T> struct Increase //函数对象：递增一个T类对象
2 { virtual void operator()(T& e) { e++; } }; //假设T可直接递增或已重载++
3
4 template <typename T> void increase(Vector<T> & V) //统一递增向量中的各元素
5 { V.traverse(Increase<T>()); } //以Increase<T>()为基本操作进行遍历
```

代码2.16 基于遍历实现`increase()`功能

■ 复杂度

遍历操作本身只包含一层线性的循环迭代，故除了向量规模的因素之外，遍历所需时间应线性正比于所统一指定的基本操作所需的时间。比如在上例中，统一的基本操作`Increase<T>()`只需常数时间，故这一遍历的总体时间复杂度为 $O(n)$ 。

§ 2.6 有序向量

若向量`S[0, n)`中的所有元素不仅按线性次序存放，而且其数值大小也按此次序单调分布，则称作有序向量（**sorted vector**）。例如，所有学生的学籍记录可按学号构成一个有序向量（学生名单），使用同一跑道的所有航班可按起飞时间构成一个有序向量（航班时刻表），第二十九届奥运会男子跳高决赛中各选手的记录可按最终跳过的高度构成一个（非增）序列（名次表）。与通常的向量一样，有序向量依然不要求元素互异，故通常约定其中的元素自前（左）向后（右）构成一个非降序列，即对任意 $0 \leq i < j < n$ 都有 $S[i] \leq S[j]$ 。

2.6.1 比较器

当然，除了与无序向量一样需要支持元素之间的“判等”操作，有序向量的定义中实际上还隐含了另一更强的先决条件：各元素之间必须能够比较大小。这一条件构成了有序向量中“次序”概念的基础，否则所谓的“有序”将无从谈起。

多数高级程序语言所提供的基本数据类型都满足上述条件，比如C++语言中的整型、浮点型和字符型等，然而字符串、复数、矢量以及更为复杂的类型，则未必直接提供了某种自然的大小比较规则。采用很多方法，都可以使得大小比较操作对这些复杂数据对象可以明确定义并且可行，比如最常见的就是在内部指定某一（些）可比较的数据项，并由此确立比较的规则。这里沿用2.5.3节的约定，假设复杂数据对象已经重载了“<”和“<=”等操作符。

2.6.2 有序性甄别

作为无序向量的特例，有序向量自然可以沿用无序向量的查找算法。然而，得益于元素之间的有序性，有序向量的查找、唯一化等操作都可更快地完成。因此在实施此类操作之前，都有必要先判断当前向量是否已经有序，以便确定是否可采用更为高效的接口。